

CURRICULUM VITAE



Miloš Vasić

AI Engineer · Software Engineer

LLM infrastructure, autonomous agents, and the governance that makes them trustworthy. Fleets over monoliths; anti-bluff, evidence-gated quality.

milos85vasic@gmail.com · milosvasic.ru · vasic.digital
GitHub: vasic-digital · HelixDevelopment · Server-Factory

2009

ENGINEERING SINCE

140+REPOSITORIES
GOVERNED**43**LLM PROVIDERS
UNIFIED**6+**

CORE LANGUAGES

AI Engineer — LLM-Infrastruktur, autonome Agenten und die Governance, die sie vertrauenswürdig macht.

- E-Mail: milos85vasic@gmail.com
- Web: <https://milosvasic.ru> · <https://vasic.digital>
- GitHub: vasic-digital · HelixDevelopment · Server-Factory

Zusammenfassung

AI/Software-Ingenieur, der AI-Entwicklungssysteme von Anfang bis Ende aufbaut – von der Multi-Provider-LLM-Infrastruktur und autonomen Agenten bis zu den QA- und Governance-Ebenen, die für ihre Zuverlässigkeit sorgen. Ich liefere keine Demos; ich liefere Plattformen. Über 15 Jahre Berufserfahrung im Ingenieurwesen (seit 2009) in den Bereichen Mobile SDKs, Echtzeit-Hardware-Integration und verteilte Backends münden nun in einen einzigen Fokus: autonome AI-Entwicklung skalierbar und vertrauenswürdig zu gestalten.

Ich entwerfe Flotten, keine Monolithen – große Produktanwendungen, die auf Dutzenden kleinen, entkoppelten und unabhängig getesteten Modulen basieren, von denen jedes eine gemeinsame technische Constitution erbt und durch eine „Anti-Bluff“-QA-Disziplin mit evidenzbasierten Prüfpunkten verifiziert wird. Hauptsprache Go, mit Kotlin/KMP, TypeScript/React, Python, Swift und Shell. Leitprinzip, mechanisch durchgesetzt statt nur proklamiert: Ein Feature gilt erst dann als fertig, wenn ein echter Nutzer es verwenden kann und es nachweisbare Belege dafür gibt.

Was ich einbringe: die Fähigkeit, eine AI-Funktionalität von einer Forschungs idee zu einem regulierten, selbstvalidierenden und produktionsreifen System zu entwickeln – LLM-Routing, das jeden Modelltest besteht, bevor man ihm vertraut, Agenten, die diskutieren und Konsens finden, statt zu raten, Speicher- und RAG-Ebenen, die den Kontext nicht verlieren, und ein ganzes Ökosystem, in dem „die Tests sind grün“ niemals stillschweigend „das Feature ist defekt“ bedeuten kann.

Kernkompetenzen

- **AI / LLM-Systeme:** Multi-Provider-LLM-Abstraktion (40+ Anbieter), MCP-Tool-Integration, RAG, vector-Datenbanken & embeddings, Agenten-Orchestrierung (kopfloze CLI-Agenten, Graph-Workflows, mehrstufige Debatten/Konsensfindung), Planung (HiPlan/MCTS/Tree-of-Thoughts), LLMops, Benchmarking (SWE-bench/HumanEval/MMLU), LLM-Verifizierung, defensive LLM-Schutzmechanismen, Computer Vision + LLM-Vision.
- **Backend-Entwicklung:** Go (Gin), gRPC + Protobuf, HTTP/3 (QUIC), WebSockets, verteilte Systeme (inkl. formale TLA+-Spezifikationen), Hochdurchsatz-REST-Dienste und parallele Worker.

- **Daten:** PostgreSQL, SQLite, SQLCipher (Verschlüsselung im Ruhezustand), Redis, Neo4j, ClickHouse, Objektspeicher (MinIO/S3/GCS/Azure).
- **Frontend / plattformübergreifend:** TypeScript/React (Tailwind, Redux Toolkit, i18next), Angular, Electron, React Native, Kotlin Multiplatform, Android/Android TV (Kotlin), iOS (Swift), Tauri/Rust.
- **Infrastruktur / DevOps:** Docker & Compose, Kubernetes + Helm, Prometheus + Grafana, OpenTelemetry, QEMU/Libvirt/Parallels; CI/CD via GitHub Actions, Gradle, Make.
- **QA / Qualitätssicherung:** „Anti-Bluff“-QA mit evidenzbasierten Prüfpunkten (HelixQA), Challenge-Harnesses mit Mutations-Gates, `go test -race`, visuelle Regressionstests, ADB-Gerätetests, SonarQube, Sicherheits-Scans (semgrep/gosec/trivy/snyk/gitleaks/nancy).
- **Engineering-Governance:** Constitution als Submodul; Vererbungs- und Propagations-Gates; Dokumentations- und Coverage-Disziplin über eine Flotte von 140+ Repositories.

Ausgewählte Projekte

Governance & QA

- **HelixConstitution** — ein universelles Ingenieursregelwerk, das als Git-Submodul verteilt und über 140+ Repositories hinweg vererbt wird: Ingenieursrecht wird exakt wie Code ausgeliefert und versionsgebunden. Ein einziger Submodul-Update aktualisiert die Regeln für die gesamte Flotte, und Ausbreitungskontrollen durchsuchen jedes nutzende Repository wortwörtlich nach der geforderten Klausel – jede Kontrolle ist mit einem Mutationstest gekoppelt, der beweist, dass die Kontrolle selbst kein leeres Versprechen ist. Aus „Best Practices, die man hoffentlich befolgt“ wird so ein vererbbares, auditierbares und maschinell durchgesetztes Anti-Bluff-Gesetz.
- **HelixQA** — Anti-Bluff-QA-Orchestrierung (Go), aufgebaut auf einer kompromisslosen Regel: Der Maßstab ist nicht „Tests bestehen“, sondern „Nutzer können die Funktion verwenden“. Es führt geschriebene YAML-Testbatterien *und* vollautonome LLM- und Vision-QA-Sitzungen aus, die die echte App öffnen, jede dokumentierte Funktion überprüfen, nach undokumentierten Fehlern auf Android/Android TV/Web/Desktop suchen und sich weigern, ein BESTANDEN zu vergeben, ohne erfasste Laufzeitbeweise – Screenshots, Logcat, Videos, Stack Traces – plus AI-fertige Tickets zur Fehlerbehebung.

AI-Entwicklung & LLM-Infrastruktur

- **HelixAgent** — ein produktionsreifer Ensemble-LLM-Dienst (Go/Gin), der keinem einzelnen Modell vertraut: Er verteilt Prompts an mehrere Anbieter, führt strukturierte Mehrunden-Debatten (Vorschlag → Kritik → Prüfung → Synthese) durch und leitet Anfragen anhand von Live-Verifizierungsergebnissen mit sanftem Fallback weiter – alles hinter einer OpenAI-kompatiblen API mit Hochverfügbarkeits-Datenschicht, Observability und Schutzmechanismen. *Go, Gin, PostgreSQL, Redis, Prometheus/Grafana/OpenTelemetry, MCP, Neo4j/ClickHouse/Kafka.*
- **HelixCode** — eine verteilte AI-Entwicklungsplattform, die Arbeit in intelligente, abhängigkeitsbewusste Aufgaben unterteilt, die auf einer von SSH verwalteten Worker-Flotte ausgeführt werden, und Checkpoints setzt sowie Rollbacks ermöglicht, sodass nichts verloren geht, wenn eine Aufgabe unterbrochen wird; hardwarebewusste Modellauswahl und vollständiger Plan/Build/Test/Refactor-Lebenszyklus hinter REST/CLI/TUI/MCP. *Go, Gin, PostgreSQL, Redis, SSH, MCP, llama.cpp/Ollama.*

- **HelixLLM** — ein Binärprogramm, sechs Bereitstellungsmodi: OpenAI- und Anthropic-kompatible Inferenz über HTTP/3, skalierbar vom Laptop bis zum Multi-Host-Cluster, mit lokaler llama.cpp-Inferenz (CUDA/Metal/ROCm) und einer automatisch erkannten, verifizierten Cloud-Fallback-Kette, die stets auf ein garantiert lokales Modell zurückfällt. *Go, HTTP/3 QUIC, gRPC/SSE/Kafka, llama.cpp.*
- **LLMProvider / LLMOrchestrator / LLMsVerifier** — das Rückgrat der LLM-Infrastruktur: eine Schnittstelle für 43 Anbieter mit Circuit Breakern, Gesundheitsüberwachung, verzögerten Wiederholungsversuchen und ehrlicher (kein hartcodierter Fallback) Modellsuche; eine thread-sichere Steuerungsebene, die headlose CLI-Agenten (OpenCode, Claude Code, Gemini, Junie, Qwen Code) über ein hybrides Pipe- und Dateiprotokoll startet und steuert; sowie eine Verifizierungsinstanz, deren verpflichtende „Siehst du meinen Code?“-Kontrolle sicherstellt, dass nur nachweislich funktionierende Modelle als nutzbar markiert oder exportiert werden.
- **HelixMemory / HelixSpecifier** — eine einheitliche Kognitions- und Gedächtnis-Engine, die vier erstklassige Backends (Mem0, Cognee, Letta, Graphiti) hinter einer einzigen Schnittstelle mit paralleler Suche und quellenübergreifender Neurangordnung vereint; sowie eine spezifikationsgetriebene Entwicklungs-Fusionsengine, die ihre eigene Zeremonie an den Arbeitsumfang anpasst und Spezifikationen durch Multi-Agenten-Debatten absichert.
- **HelixTrack** — eine freiheitliche JIRA- + Confluence-Alternative (Flaggschiff der Helix-Track-Reihe): Go-Mikrodienste mit einer einheitlichen, aktionsgesteuerten API über HTTP/3, SQLCipher-Verschlüsselung im Ruhezustand und nativen Web-/Desktop-/Android-/iOS-Clients.

Produktreife Tools (vasic-digital utils)

- **Catalogizer** — Multiprotokoll-fähiges, verschlüsseltes, selbstgehostetes Medienverwaltungssystem (Go/Gin + React), robust gegenüber instabilen Netzwerkspeichern, aufgebaut auf 21 wiederverwendbaren Submodulen.
- **Courses-Creator** — Markdown-zu-Video-AI-Kurs-Pipeline mit TTS sowie Desktop-, Mobile- und Web-Playern.
- **VisionEngine** — Computer-Vision + Multi-Provider-LLM-Vision-UI-Wahrnehmung mit Navigationsgraphen.
- **DocProcessor · Docs Chain · Herald · task_bridge · Vasic Digital Wiederverwendbare Modulsuite** — QA-Feature-Mapping, inhaltsgehashtes Dokumenten-/Datenbanksynchronisation, Benachrichtigungen in natürlicher Sprache, Aufgaben-/Board-Synchronisation sowie die `digital.vasic.*`-Standardbibliotheksflotte.

Infrastrukturautomatisierung (Server Factory)

- **Mail Server Factory** — Deklarative JSON → vollständig provisionierte, dockerisierte Mailserver für 12 Verbindungstypen und 25 Linux-Distributionen; meldet 439 bestandene Tests und ein sauberes SonarQube-Gate.
- **Server Factory Core Framework, Qemu-Utills, Parallels-Utills** — das gemeinsame Provisionierungs-Backend und VM-Image-Tooling.

Sprachen & Tools (Kurzübersicht)

Go · Kotlin · Kotlin Multiplatform · TypeScript · JavaScript · Python · Swift · Java · Rust · Shell · PL/pgSQL · TLA+ · Gin · gRPC · HTTP/3 · React · Angular · Electron · React Native · PostgreSQL · SQLite · SQLCipher · Redis · Neo4j · ClickHouse · Docker · Kubernetes · Prometheus · Grafana · OpenTelemetry · QEMU · GitHub Actions · Gradle · Make

Berufserfahrung

Seit 2009 als Softwareentwickler tätig – über den gesamten Entwicklungszyklus hinweg: Planung, Entwicklung, Teamleitung und Deployment. Die vollständige Historie basiert auf dem verifizierten Nachweis des Kandidaten (milosvasic.ru).

Festanstellungen

- **SDK-Entwickler — Harness** (harness.io), Belgrad, Serbien · 03/2020 – 12/2024. Leitender Entwickler für die SDK-Familie der Feature-Flag-Abteilung des Unternehmens, mit Fokus auf alle wichtigen mobilen Plattformen und darüber hinaus. Kunden und Partner umfassten AWS, Google sowie verschiedene Banken. *Technologien: Android, iOS, Flutter, React Native, TypeScript, JavaScript, Java, Kotlin, Swift, Go, Ruby.*
- **Softwareentwickler — Leica Geosystems** (leica-geosystems.com), Heerbrugg, Schweiz · 02/2016 – 02/2020. Hauptsächlich iOS- und Android-Entwicklung für die hochmodernen 3D-Scanner von Leica Geosystems – Echtzeitkommunikation mit der Hardware, Datenverarbeitung und Synchronisation. Partner: Autodesk. *Technologien: Android, iOS, Java, Kotlin, Swift, C++.*
- **SDK-Entwickler — Bosch** (bosch.rs), Belgrad, Serbien · 01/2010 – 01/2016. Leitender SDK-Entwickler für das Connected-Vehicles-SDK-Projekt – Echtzeit-Bluetooth-Kommunikation mit dem OBD2-Bus, Hochleistungs-Datenverarbeitung und -persistenz. *Technologien: Android, Java, Kotlin.*

Weitere Tätigkeiten

- **TN-TECH** (tn-tech.co.rs), Novi Sad, Serbien · Teilzeit, seit 03/2017. Arbeit für Globex Data (Kanada und Schweiz) — Sekur (SekurMessenger), SekurMail, SekurSuite — sowie die BusRide-Plattform. *Technologien: Android, Java, Kotlin, C++, Qt.*
- **Increment Loop** (incrementloop.com), Belgrad, Serbien · Teilzeit, seit 09/2023. Die Yuno-Anwendung. *Technologien: Android, Kotlin.*
- **Open-Source / eigene Organisationen** — HelixTrack, Server Factory (Mail Server Factory, Parallels-Utills, Qemu-Utills) und Vasic Digital (Android-Toolkit, Network-Binder), näher beschrieben unter „Ausgewählte Projekte“ oben.

Veröffentlichungen

- **Grundlagen der Kotlin** — Selbstverlag; letzte überarbeitete Auflage September 2022 (*Grundlagen der Kotlin*, 3. Auflage). Zudem Autor für Packt Publishing (UK).

Ausbildung

- **M.Sc., Moderne Informationstechnologien** — Universität Singidunum, Belgrad, Serbien · 2014.
- **B.Sc., Informatik und Computing** — Universität Singidunum, Belgrad, Serbien · 2008.